Algebraic structures of the Lindblad equation

Examples 1 and 2

Leonel Bixano
Centro de Investigación y de Estudios Avanzados del Instituto Politécnico Nacional,
Departamento de Física,

Guillermo López-Alvarez, Victor Alberto Cruz-Barriguete, Victor Guadalupe Ibarra-Sierra, José Luis Cardoso, Juan Carlos Sandoval-Santana, Alejandro Kunold
Área de Física Teórica y Materia Condensada
Universidad Autónoma Metropolitana

Abstract

This Mathematica notebook accompanies the paper Algebraic Structures of the Lindblad Equation and presents five worked examples illustrating the algebraic formulation of finite-dimensional Lindblad dynamics.

Three examples verify the derived algebraic relations for one-, two-, and three-qubit operator bases. The calculations proceed from the construction of an orthonormal Hermitian operator basis and its structure constants to the associated superoperator algebra. The resulting Liouville superoperators and differential equations for the dynamical-map coefficients are compared with those obtained by direct construction, confirming their agreement.

Two numerical examples describe transmons truncated to their two and four lowest energy levels, including coherent driving, energy relaxation, and pure dephasing. They solve the dynamical-map equations and illustrate the resulting population dynamics. The four-level example demonstrates an approximate X gate and population leakage beyond the computational subspace. Both examples construct the unnormalized Choi matrix to check Hermiticity preservation, complete positivity, and trace preservation of the computed maps. Additional checks in the four-level example examine purely dissipative evolution with the Hamiltonian set to zero and unitary evolution with the Kossakowski matrix set to zero.

The algebraic construction is parameterized by the number of qubits and includes recursion relations for extending it to larger operator bases. Although symbolic calculations and storage requirements become demanding as the system size increases, the implementation provides a framework that can be adapted to other finite-dimensional open quantum systems.

Licence

LindbladAlgebra Mathematica examples accompanying the paper

“The Algebraic Structure of the Lindblad Equation”

Copyright (c) 2026 Alejandro Kunold and collaborators

This notebook is distributed under the MIT License. Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, subject to the conditions of the MIT License.

A copy of the full license is available in the LICENSE file of this repository.

In this section we create a series of Mathematica functions that facilitate the computation of the matrix basis, structure constants and Liouville superoperator.

Instructions

Run the cell Functions below before running any example. Running Example 2 requires running first Example 1 in order to have the needed mathematical expressions. Running Example 4 requires running first Example 3

Functions

In this section we create a series of Mathematica functions that facilitate the computation of the matrix basis, structure constants and Liouville superoperator.

Functions

Example 1. One qubit.

One-qubit matrix basis h

In the following example we verify the main relations derived in the paper for the one qubit case.

In[•]:=
```
nq = 1;
```

We begin by constructing the one-qubit basis $\mathbf{h}_n$ and its associated structure constant superoperators $\boldsymbol{Z}$, $\boldsymbol{Z}^*$ and $\boldsymbol{X}$.

In[•]:=
```
h = QubitBasis[nq];
{b, c} = BCStructureConstantsFromQubits[nq];
z = ZStructureConstantsFromQubits[nq];
zt = Conjugate[z];
x = XBasisFromQubits[nq];
```

The complex structure-constant superoperators can be visualized with the following line of code.

In[•]:=
```
MatrixForm[z[[3]]]
```

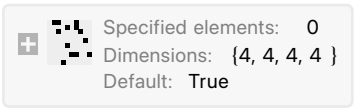
Out[•]//MatrixForm=

$$\begin{pmatrix} 0 & 0 & \sqrt{2} & 0 \\ 0 & 0 & 0 & -i\sqrt{2} \\ \sqrt{2} & 0 & 0 & 0 \\ 0 & i\sqrt{2} & 0 & 0 \end{pmatrix}$$

It can be verified that $\boldsymbol{Z}$ and $\boldsymbol{Z}^*$ constitute to independent realizations of the Lie algebra associated with h by showing that the following difference, obtained by subtracting the right-hand side from the left-hand side of Eqs. (70) and (71), vanish identically.

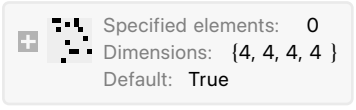
In[•]:=
```
zz = Transpose[z.Transpose[z], {1, 3, 2, 4}];
commu = SparseArray[zz - Transpose[zz]];
PossibleZeroQ[SparseArray[commu + 2 i c.z]]
```

Out[•]=

SparseArray[]

In[•]:=
```
ztzt = Transpose[zt.Transpose[zt], {1, 3, 2, 4}];
commu = SparseArray[ztzt - Transpose[ztzt]];
PossibleZeroQ[SparseArray[commu - 2 i c.zt]]
```

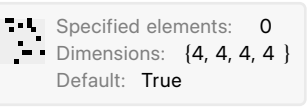
Out[•]=

SparseArray[]

Similarly we can verify that $\boldsymbol{Z}$, $\boldsymbol{Z}^*$ commute.

```
In[•]:= zzt = Transpose[z.ConjugateTranspose[z]];
        ztz = Transpose[zt.Transpose[z], {3, 1, 2, 4}];
        commu = SparseArray[zzt - ztz];
        PossibleZeroQ[commu]
```

```
Out[•]=
```

SparseArray[]

The structure of the 16 elements of $\boldsymbol{X}$ can be visualized using the following line of code.

```
In[•]:= MatrixForm[x[[2, 3]]]
```

```
Out[•]//MatrixForm=
```

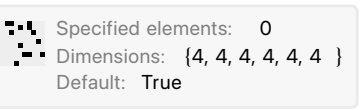
$$\begin{pmatrix} 0 & 0 & 0 & \frac{i}{2} \\ 0 & 0 & \frac{1}{2} & 0 \\ 0 & \frac{1}{2} & 0 & 0 \\ -\frac{i}{2} & 0 & 0 & 0 \end{pmatrix}$$

The Lie and Jordan algebraic relations of $\boldsymbol{X}$ can be verified by evaluating the following differences, obtained by subtracting the right-hand side from the left-hand side of Eqs. (82) and (83), and verifying that they vanish identically. This calculation may exceed the available memory if $n_q \geq 2$.

```
In[•]:= xx = Transpose[x.Transpose[x, {2, 3, 1, 4}], {1, 2, 5, 3, 4, 6}];
        commu = SparseArray[xx - Transpose[xx, {3, 4, 1, 2, 5, 6}]];
        anticommu = SparseArray[xx + Transpose[xx, {3, 4, 1, 2, 5, 6}]];
```

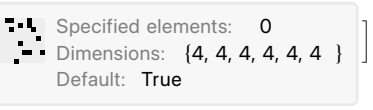
```
In[•]:= PossibleZeroQ[SparseArray[commu -
          1/4 Transpose[(zt.Transpose[x, {1, 4, 2, 3}].z - z.Transpose[x, {1, 4, 2, 3}].zt),
            {1, 3, 5, 6, 2, 4}]]]
```

```
Out[•]=
```

SparseArray[]


```
In[•]:= PossibleZeroQ[SparseArray[anticommu -
          1/4 Transpose[(zt.Transpose[x, {1, 4, 2, 3}].z + z.Transpose[x, {1, 4, 2, 3}].zt),
            {1, 3, 5, 6, 2, 4}]]]
```

```
Out[•]=
```

SparseArray[]

The orthonormality of $\mathfrak{X}_{n^2}$ is verified by flattening the first two indices of $\boldsymbol{X}$ into a single index and then evaluating the Hilbert-Schmidt inner product

In[•]:=
```
X = Flatten[x, 1]
```

Out[•]=
```
SparseArray[  Specified elements: 64
Dimensions: {16, 4, 4} ]
```

In[•]:=
```
XX = SparseArray[TensorContract[X.Transpose[X], {{2, 4}}]]
DiagonalMatrixQ[XX]
```

Out[•]=
```
SparseArray[  Specified elements: 16
Dimensions: {16, 16} ]
```

Out[•]=
```
True
```

Hamiltonian

A convenient system for illustrating the results developed in this work is the transmon qubit. Since the number of quantum levels retained in the model can be chosen arbitrarily, it provides a simple framework for studying the behavior of the algebraic structures introduced above as the Hilbert-space dimension increases. We first define the lowering and rasing operators as well as the jump operators

In[•]:=
```
m = 2^nq;
n = 2^(2 nq);
a = SparseArray[Table[{n + 1, n + 2} → Sqrt[n + 1], {n, 0, m - 2}], m];
at = SparseArray[Table[{n + 1, n} → Sqrt[n], {n, 1, m - 1}], m];
A = SparseArray[Table[{n + 1, n + 1, n + 2} → Sqrt[n + 1], {n, 0, m - 2}], {m - 1, m, m}];
At = SparseArray[Table[{n + 1, n + 2, n + 1} → Sqrt[n + 1], {n, 0, m - 2}], {m - 1, m, m}];
```

The Hamiltonian in the interaction picture, given by Eq. (129) is calculated using the following commands

In[•]:=
```
ψ = Exp[-i (Δ t + ϕ)] Map[Exp, i α t (1 - Range[m - 1])];
ψt = Exp[i (Δ t + ϕ)] Map[Exp, -i α t (1 - Range[m - 1])];
Hi = Ω/2 (ψ.A + ψt.At);
MatrixForm[Hi]
```

Out[•]//MatrixForm=
$$\begin{pmatrix} 0 & \frac{1}{2} e^{-i (t \Delta+\phi)} \Omega \\ \frac{1}{2} e^{i (t \Delta+\phi)} \Omega & 0 \end{pmatrix}$$

The Hamiltonian vector is obtained by projecting the Hamiltonian on the elements of the set $\mathbf{h}_n$ through Eq. (23).

```
In[•]:= H = ExpToTrig[ProjectOperatorOntoBasis[Hi, h]];
        MatrixForm[H]
```

Out[•]//MatrixForm=

$$\begin{pmatrix} 0 \\ \frac{\Omega \text{Cos}[t\,\Delta+\phi]}{\sqrt{2}} \\ \frac{\Omega \text{Sin}[t\,\Delta+\phi]}{\sqrt{2}} \\ 0 \end{pmatrix}$$

Lindbladian

To construct the Lindbladian, we first introduce the corresponding jump operators. In the case of transmons, two decoherence processes are particularly relevant: energy relaxation and pure dephasing. Energy relaxation describes the loss of an excitation from the transmon to its environment and is represented by the Lindblad jump operator $J_1 = a$. This process is characterized by the relaxation time T_1, time or equivalently by the relaxation rate $\Gamma_1 = 1/T_1$. Although the corresponding contribution to the Lindbladian is time independent in the Schrödinger picture, it becomes time dependent in the interaction picture used here to express the Hamiltonian. For a harmonic oscillator a simple scalar phase cancels completely, but for the anharmonic transmon the phase contains the number operator n=$a^\dagger a$, that introduces the time dependency. This time dependency is easily introduced through the phases and decomposing into transition operators the lowering and rising operators

```
In[•]:= J1 = SparseArray[Table[{n, n, n + 1} → √n Exp[-i α n t], {n, 1, m - 1}], {m - 1, m, m}];
        J1t = SparseArray[Table[{n, n + 1, n} → √n Exp[i α n t], {n, 1, m - 1}], {m - 1, m, m}];
```

The Lindbladian corresponding to the energy relaxation part is defined by Eq. (38)

```
In[•]:= γ1 = SparseArray[Flatten[Table[{ii, jj} → γ10, {ii, m - 1}, {jj, m - 1}]], {m - 1, m - 1}];
        Γ1 = GammaFromgamma[h, γ1, J1, J1t];
        MatrixForm[Γ1]
```

Out[•]//MatrixForm=

$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & \frac{\gamma 10}{4} & \frac{i\,\gamma 10}{4} & 0 \\ 0 & -\frac{i\,\gamma 10}{4} & \frac{\gamma 10}{4} & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Pure dephasing describes a phenomenon that does not change the energy-level populations but instead, destroys coherence between different energy eigenstates. It is characterized by the jump operator $J_2 = a^\dagger a$

```
In[•]:= J2 = SparseArray[{at.a}];
        J2t = SparseArray[{at.a}];
```

and the pure-dephasing time T_2 or rate $\Gamma_2 = 1/T_1 - 1/2\,T_2$ which is associated

```
In[•]:= γ2 = SparseArray[{{γ20}}];
        Γ2 = GammaFromgamma[h, γ2, J2, J2t];
        MatrixForm[Γ2]
```

Out[•]//MatrixForm=

$$\begin{pmatrix} \frac{\gamma 20}{4} & 0 & 0 & -\frac{\gamma 20}{4} \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ -\frac{\gamma 20}{4} & 0 & 0 & \frac{\gamma 20}{4} \end{pmatrix}$$

The full Lindbladian is therefore

```
In[•]:= Γ = Γ1 + Γ2;
        MatrixForm[Γ]
```

Out[•]//MatrixForm=

$$\begin{pmatrix} \frac{\gamma 20}{4} & 0 & 0 & -\frac{\gamma 20}{4} \\ 0 & \frac{\gamma 10}{4} & \frac{i\,\gamma 10}{4} & 0 \\ 0 & -\frac{i\,\gamma 10}{4} & \frac{\gamma 10}{4} & 0 \\ -\frac{\gamma 20}{4} & 0 & 0 & \frac{\gamma 20}{4} \end{pmatrix}$$

Algebraic Method

Having obtained the quantities required to construct the Hamiltonian and Lindbladian contributions to the Liouville superoperator, namely ***H*** and Γ, we can proceed to compute Λ using Eq. (60), yielding

```
In[•]:= Λ = SparseArray[Simplify[Normal[LambdaFromHamiltonianLindbladian[z, H, Γ]]]];
        MatrixForm[Λ]
```

Out[•]//MatrixForm=

$$\begin{pmatrix} \frac{1}{4}\,(-2\,\gamma 10 - \gamma 20) & -i\,\Omega\,\text{Cos}[t\,\Delta + \phi] & -i\,\Omega\,\text{Sin}[t\,\Delta + \phi] & \frac{\gamma 10}{4} \\ i\,\Omega\,\text{Cos}[t\,\Delta + \phi] & \frac{\gamma 10}{4} & \frac{i\,\gamma 10}{4} & 0 \\ i\,\Omega\,\text{Sin}[t\,\Delta + \phi] & -\frac{i\,\gamma 10}{4} & \frac{\gamma 10}{4} & 0 \\ \frac{\gamma 10}{4} & 0 & 0 & \frac{\gamma 20}{4} \end{pmatrix}$$

The Liouville superoperator is obtained using Eq. (59).

```
In[•]:= Lam = SparseArray[Simplify[Normal[LiouvilleSOFromLamda[x, Λ]]]];
        MatrixForm[Lam]
```

Out[•]//MatrixForm=

$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & \frac{1}{4}\,(-\gamma 10 - \gamma 20) & 0 & \Omega\,\text{Sin}[t\,\Delta + \phi] \\ 0 & 0 & \frac{1}{4}\,(-\gamma 10 - \gamma 20) & -\Omega\,\text{Cos}[t\,\Delta + \phi] \\ \frac{\gamma 10}{2} & -\Omega\,\text{Sin}[t\,\Delta + \phi] & \Omega\,\text{Cos}[t\,\Delta + \phi] & -\frac{\gamma 10}{2} \end{pmatrix}$$

Direct method

The Liouville superoperator is also computed in the same matrix basis for comparison. The two meth-

ods are found to yield identical results.

```
In[•]:= Ldm = SparseArray[Simplify[Normal[LiouvilleSOFromProjection[h, H, Γ]]]];
        MatrixForm[Ldm]
```

Out[•]//MatrixForm=

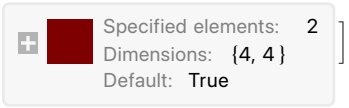
$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & \frac{1}{4}(-\gamma 10 - \gamma 20) & 0 & \Omega \, \text{Sin}[t\,\Delta + \phi] \\ 0 & 0 & \frac{1}{4}(-\gamma 10 - \gamma 20) & -\Omega \, \text{Cos}[t\,\Delta + \phi] \\ \frac{\gamma 10}{2} & -\Omega \, \text{Sin}[t\,\Delta + \phi] & \Omega \, \text{Cos}[t\,\Delta + \phi] & -\frac{\gamma 10}{2} \end{pmatrix}$$

Comparison of the algebraic method and the direct method

To compare the Liouville superoperators obtained using the algebraic and direct methods, we subtract one superoperator from the other.

```
In[•]:= PossibleZeroQ[SparseArray[Lam - Ldm]]
```

Out[•]=

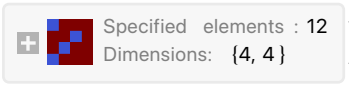
SparseArray[]

Comparison of the algebraic method with the standard Kronecker product method

To compare the Liouville superoperators obtained using the algebraic and standard Kronecker product methods, we first construct the Kronecker representation using Eqs. (21) and (22).

```
In[•]:= ℐ = SparseArray[IdentityMatrix[m]];
        J1T = SparseArray[Table[{n, n + 1, n} → √n Exp[-i α n t], {n, 1, m - 1}], {m - 1, m, m}];
        J1s = SparseArray[Table[{n, n, n + 1} → √n Exp[i α n t], {n, 1, m - 1}], {m - 1, m, m}];
        J2T = Transpose[J2, {1, 3, 2}];
        J2s = Conjugate[J2];
        Lsme = LiouvilleStandardKronecker[Hi, ℐ, γ1, J1, J1t, J1s, J1T, γ2, J2, J2t, J2s, J2T]
```

Out[•]=

SparseArray[]

We then transform it from the matrix basis $\mathbf{e}_n$, defined in Eq. (12), to the basis $\mathbf{h}_n$ using the unitary transformation $u_{i,j}$=tr[$h_i e_j$]

```
In[•]:= e = ElementaryBasis[m];
        u = TensorContract[h.Transpose[e], {{2, 4}}];
        ut = ConjugateTranspose[u];
        Lsmh = u.(Lsme.ut)
```

Out[•]=

SparseArray [Specified elements: 16 Dimensions: {4, 4}]

To compare the Liouville superoperators obtained using the algebraic and standard Kronecker product methods, we subtract one superoperator from the other.

```
In[•]:= PossibleZeroQ[SparseArray[Lam - Lsmh]]
```

Out[•]=

SparseArray [Specified elements: 8 Dimensions: {4, 4} Default: True]

Liouville equation for the dynamical map

To verify that the dynamical differential equations for the coordinate vector, Eq. (113), are equivalent to those derived from the Lindbladian, we directly project the Liouville operator acting on the dynamical map, as given on the right-hand side of Eq. (110), onto the basis $\boldsymbol{X}$. We then subtract the resulting expression from the right-hand side of Eq. (113), which should yield zero if both formulations are equivalent. First, we define the elements of the dynamical map as in Eq. (108).

```
In[•]:= V = SparseArray[Array[v[##][t] &, {n, n}]];
```

We calculate the right-hand side of the dynamical differential equations using the algebraically derived Eq. (113).

```
In[•]:= DiffEqsAlgebraicMethod =
          SparseArray[Flatten[V].Flatten[Transpose[SparseArray[1/4 zt.Λ.z], {1, 3, 2, 4}], 1]]
```

Out[•]=

SparseArray [Specified elements: 16 Dimensions: {4, 4}]

Here, we project the Liouville operator obtained through the direct method, acting on the dynamical map as given on the right-hand side of Eq. (110), onto the basis $\boldsymbol{X}$.

```
In[•]:= DiffEqsDirectMethod = TensorContract[
          Transpose[V].TensorContract[x.Transpose[x, {2, 3, 1, 4}].Ldm, {{3, 6}}], {{1, 2}}]
```

Out[•]=

SparseArray [Specified elements: 16 Dimensions: {4, 4}]

Finally, we compare the results obtained by both methods.

In[•]:=
```
PossibleZeroQ[SparseArray[DiffEqsDirectMethod - DiffEqsAlgebraicMethod]]
```

Out[•]=

SparseArray [Specified elements: 16
Dimensions: {4, 4}
Default: True]

Example 2. Two-level transmon dynamics: *X*-gate implementation and leakage

Numerical solution of the differential equation

To illustrate how to solve the resulting system of ordinary differential equations, we study the dynamics of a transmon truncated to its four lowest energy levels, using a pulse designed to implement an *X* (*NOT*) gate. This allows us to examine population leakage from the computational subspace, spanned by $|0\rangle$ and $|1\rangle$, into the higher levels $|2\rangle$ and $|3\rangle$ during the gate operation. For this purpose we use the differential equations derived in the previous example (stored in the variable DiffEqsAlgebraicMethod). We flatten the time derivative of the dynamical map *V(t)* and equate it to the flattened right-hand side obtained above. This process yields the equations in the format required by NDSolve.

In[•]:=
```
fV = Flatten[V];
eqs = Thread[D[fV, t] == Flatten[DiffEqsAlgebraicMethod]];
```

We choose parameter values typical of transmon qubits: $\Omega_0/2\pi$ = 50MHz, $\alpha/2\pi$ = -300MHz, $\omega/2\pi$ = 3 GHz, T_1 = 150 μs and T_2 = 100 μs.

In[•]:=
```
Ω0 = 2 π 0.05;
α0 = -2 π 0.3;
ω0 = 2 π 3.0;
T1 = 150.0 × 10^3 ;
T2 = 100.0 × 10^3;
Γ10 = 2 / T1;
Γ20 = 4/T2 - 2/T1;
```

Additionally, to facilitate comparison with known results, we use a Gaussian pulse with width $\sigma = 2\pi/\Omega_0$ and choose the phase ϕ to implement an X gate. We then construct the system of equations, stored in eqsn, using these parameter values.

In[•]:=
```
σ = 2 π / Ω0;
t0 = 4 σ;
tmin = 0;
tmax = 8 σ;
Δ0 = 0.0;
ϕ0 = 0;
constants =
  {α → α0, Δ → Δ0, Ω → Ω0 / (2 √(2 π)) Exp[-1/2 ((t - t0)/σ)^2], ϕ → ϕ0, ɣ10 → Γ10, ɣ20 → Γ20};
eqsn = Chop[eqs /. constants];
```

We impose the initial condition *V(0)=I* . In terms of the expansion coefficients, this corresponds to $V_{i,j} = 0$ for *(i,j)* ≠ *(1,1)* and $V_{1,1}$= 4, since $X_{1,1}$=I/4 for nq=2

In[•]:=
```
initialcond = Thread[(Normal[fV] /. {t → 0}) == SparseArray[{1} → 2, n^2]];
```

We combine the differential equations with the initial conditions and solve the resulting initial-value problem using NDSolve.

In[•]:=
```
diffeqsn = Join[eqsn, initialcond];
sol = NDSolve[diffeqsn, Normal[fV], {t, tmin, tmax}][[1]];
```

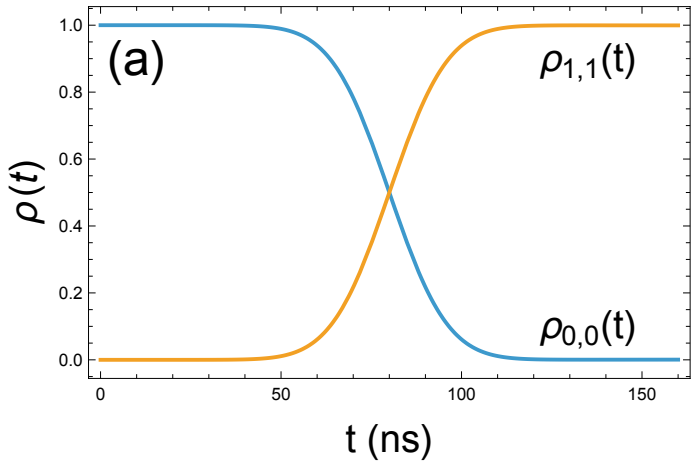
To obtain a particular case, we can set the transmon to be initially in the first state. Then, the density matrix can be expanded through Eq. (16) and the actual density matrix is recovered as expanding it in terms of the basis $\mathfrak{h}_n$. We first try the case where the lowest level is fully populated and the remaining ones are empty

In[•]:=
```
ρ0 = ProjectOperatorOntoBasis[SparseArray[{1, 1} → 1, {m, m}], h];
ρ = fV.X.ρ0;
rho = Normal[ρ.h] /. sol;
```

Plotting the populations $\rho_{1,1}(t)$ and $\rho_{2,2}(t)$ of the two first quantum levels of the transmon as a function of time

In[ ]:=
```
Show[
 Plot[{Re[rho[[1, 1]]], Re[rho[[2, 2]]]}, {t, tmin, tmax}, Axes → False,
  Frame → True, FrameLabel → {Style["t (ns)", 20], Style["ρ(t)", Italic, 20]}],
 Graphics[{
   Text[Style["ρ1,1(t)", 20], {135, 0.9}],
   Text[Style["ρ0,0(t)", 20], {135, 0.1}],
   Text[Style["(a)", 25], {10, 0.9}]
  }]
]
```

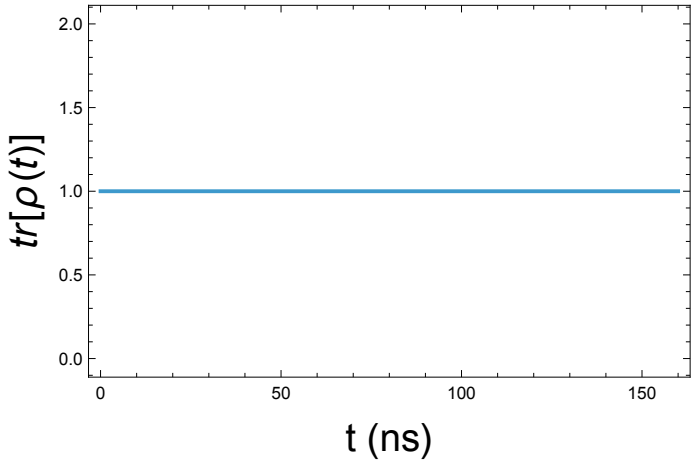
Out[ ]=



An easy way to verify that there is trace preservation we plot the trace of the density matrix as a function of time

In[ ]:=
```
Plot[Tr[rho], {t, tmin, tmax}, Axes → False, Frame → True, PlotRange → All,
 FrameLabel → {Style["t (ns)", 20], Style["tr[ρ(t)]", Italic, 20]}]
```

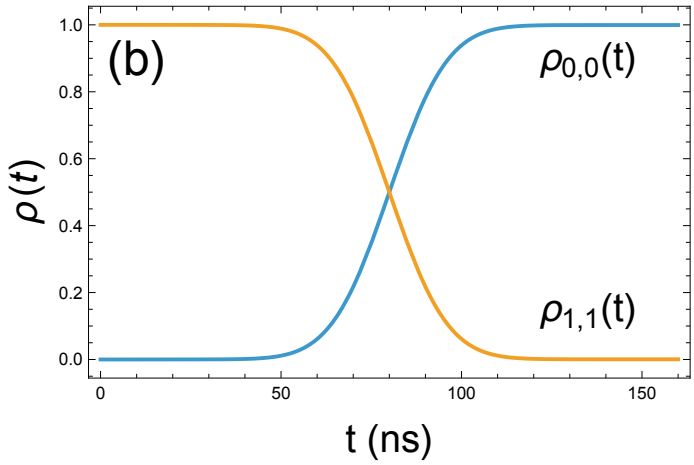
Out[ ]=



We can also try other initial configurations, as one in which the second level is initially fully populated.

```
In[•]:= ρ0 = ProjectOperatorOntoBasis[SparseArray[{2, 2} → 1, {m, m}], h];
        ρ = fV.X.ρ0;
        rho = Normal[ρ.h] /. sol;
        Show[
         Plot[{Re[rho[[1, 1]]], Re[rho[[2, 2]]]}, {t, tmin, tmax}, Axes → False,
          Frame → True, FrameLabel → {Style["t (ns)", 20], Style["ρ(t)", Italic, 20]}],
         Graphics[{
           Text[Style["ρ1,1(t)", 20], {135, 0.15}],
           Text[Style["ρ0,0(t)", 20], {135, 0.9}],
           Text[Style["(b)", 25], {10, 0.9}]
          }]
        ]
```

Out[•]=



Dynamical map of the *X* gate

The preceding results suggest behavior consistent with an X gate. To assess how closely the evolution approximates this gate, we compute the dynamical map projected onto the computational subspace spanned by the two lowest energy levels, $|0\rangle$ and $|1\rangle$. We start by computing the dynamical mat in the Liouville space and evaluate it at t = tmax.

```
In[•]:= Vmat = Normal[fV.X] /. sol /. {t → tmax};
```

The dynamical map of the *X* gate takes the form

```
In[•]:= MatrixForm[Chop[Vmat, 1 × 10^-5]]
```

Out[•]//MatrixForm=

$$\begin{pmatrix} 1. & 0 & 0 & 0 \\ 0 & 0.998401 & 0 & 0 \\ -0.0002807 & 0 & -0.998498 & -0.000198776 \\ 0 & 0 & 0.000198776 & -0.998837 \end{pmatrix}$$

This result can be compared with the actual dynamical map of the gate *X*.

```
In[•]:= U = PauliMatrix[1];
        MatrixForm[TensorContract[(h.U).Transpose[h.ConjugateTranspose[U]], {{2, 4}}]]
```

Out[•]//MatrixForm=

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}$$

This result indicates that the implemented operation approximates an X gate.

Verification of dynamical map properties using the unnormalized Choi matrix

To verify Hermiticity preservation, complete positivity, and trace preservation of the dynamical map, we construct its unnormalized Choi matrix J_t using Eq. (114) and check the corresponding conditions.

```
In[•]:= B = Flatten[h, {{3, 2}, {1}}];
        Jt = B.Transpose[V].ConjugateTranspose[B];
```

To check the three conditions, we evaluate the unnormalized Choi operator at a selected time between tmin and tmax

```
In[•]:= Jtn = Normal[Jt] /. sol /. {t → tmin + 0.8 (tmax - tmin)};
```

First, we check Hermiticity preservation by subtracting $J_t^\dagger$ from J_t

```
In[•]:= Max[Abs[Flatten[Jtn - ConjugateTranspose[Jtn]]]]
```

Out[•]=

$$1.11022 \times 10^{-16}$$

This result shows that the maximum difference between the two operators is negligible. To verify complete positivity, we calculate the eigenvalues of J_t and check that they are nonnegative within numerical precision.

```
In[•]:= Chop[Eigenvalues[Jtn]]
```

Out[•]=

```
{1.99829, 0.000811981, 0.000569596, 0.000324156}
```

Finally, we check trace preservation by taking the trace

```
In[•]:= g = KroneckerProduct[h, SparseArray[{{1, 1, 1} → 1, {1, 2, 2} → 1}, {1, 2, 2}]];
        MatrixForm[Chop[TensorContract[g.Jtn, {{2, 3}}].h]]
```

Out[•]//MatrixForm=

$$\begin{pmatrix} 1. & 0 \\ 0 & 1. \end{pmatrix}$$

These tests can also be performed in two special cases: (i) with the drive turned off; and (ii) with the Kossakowski matrix set to zero, yielding a unitary channel.

(i) drive turned off

```
In[•]:= Ω0 = 2 π 0.05;
        α0 = -2 π 0.3;
        ω0 = 2 π 3.0;
        T1 = 150.0 × 10^3 ;
        T2 = 100.0 × 10^3;
        Γ10 = 2 / T1;
        Γ20 = 4/T2 - 2/T1;
        σ = 2 π / Ω0;
        t0 = 4 σ;
        tmin = 0;
        tmax = 8 σ;
        Δ0 = 0.0;
        ϕ0 = 0;
        constants = {α → α0, Δ → Δ0, Ω → 0, ϕ → ϕ0, γ10 → Γ10, γ20 → Γ20};
        eqsn = Chop[eqs /. constants];
        initialcond = Thread[(Normal[fV] /. {t → 0}) == SparseArray[{1} → 2, n^2]];
        diffeqsn = Join[eqsn, initialcond];
        sol = NDSolve[diffeqsn, Normal[fV], {t, tmin, tmax}][[1]];
        Jtn = Normal[Jt] /. sol /. {t → tmin + 0.5 (tmax - tmin)};
```

Hermiticity

```
In[•]:= Max[Abs[Flatten[Jtn - ConjugateTranspose[Jtn]]]]
Out[•]=
        1.11022 × 10^-16
```

Positivity

```
In[•]:= Chop[Eigenvalues[Jtn]]
Out[•]=
        {1.99893, 0.000533191, 0.000533049, 0}
```

Trace preservation

```
In[•]:= g = KroneckerProduct[SparseArray[{{1, 1, 1} → 1, {1, 2, 2} → 1}, {1, 2, 2}], h];
        MatrixForm[Chop[TensorContract[g.Jtn, {{2, 3}}].h]]
Out[•]//MatrixForm=
        ( 1.00053     0
            0     0.999467 )
```

(ii) Kossakowski matrix set to zero

```
In[•]:= Ω0 = 2 π 0.05;
        α0 = -2 π 0.3;
        ω0 = 2 π 3.0;
        σ = 2 π / Ω0;
        t0 = 4 σ;
        tmin = 0;
        tmax = 8 σ;
        Δ0 = 0.0;
        ϕ0 = 0;
        constants = {α → α0, Δ → Δ0, Ω → Ω0/(2 √(2 π)) Exp[-1/2 ((t - t0)/σ)^2], ϕ → ϕ0, γ10 → 0, γ20 → 0};
        eqsn = Chop[eqs /. constants];
        initialcond = Thread[(Normal[fV] /. {t → 0}) == SparseArray[{1} → 2, n^2]];
        diffeqsn = Join[eqsn, initialcond];
        sol = NDSolve[diffeqsn, Normal[fV], {t, tmin, tmax}][[1]];
        Jtn = Normal[Jt] /. sol /. {t → tmin + 0.5 (tmax - tmin)};
```

Hermiticity

```
In[•]:= Max[Abs[Flatten[Jtn - ConjugateTranspose[Jtn]]]]
Out[•]=
        0.
```

Positivity

```
In[•]:= Chop[Eigenvalues[Jtn]]
Out[•]=
        {2., 7.00382 × 10^-8, 0, 0}
```

Trace preservation

```
In[•]:= g = KroneckerProduct[SparseArray[{{1, 1, 1} → 1, {1, 2, 2} → 1}, {1, 2, 2}], h];
        MatrixForm[Chop[TensorContract[g.Jtn, {{2, 3}}].h]]
Out[•]//MatrixForm=
        (1.  0
         0   1.)
```